The RAG Cheat Sheet

Every layer, every option, what each is good for, and which tools do it

Reference document. Not for reading front-to-back — for looking things up.

How to use this

Part 1 walks every layer of a RAG system. For each layer: the options, what each is good for, when not to use it, and the real tools.

Part 2 is the maturity matrix — what you actually pick at each layer depending on whether you're building Basic, Advanced, Hybrid, or Agentic RAG.

Part 3 covers the layers that only exist in agentic systems.

Part 4 is the "commonly missed" list — layers that are not optional but get skipped anyway.

Part 5 is quick-pick decision trees: "my problem is X, use Y."

The full layer stack (the map)

```
INDEXING PATH (offline)                    SERVING PATH (online)
──────────────────────                     ─────────────────────
 L0  Sources & connectors                   L9   Query understanding
 L1  Parsing & extraction                   L10  Retrieval
 L2  Cleaning & normalisation               L11  Fusion
 L3  Metadata & ACL capture                 L12  Reranking
 L4  Chunking                               L13  Context assembly & compression
 L5  Chunk enrichment                       L14  Generation
 L6  Embedding                              L15  Post-generation verification
 L7  Compression & quantisation
 L8  Indexes & storage

CROSS-CUTTING (touch every layer above)
 L16  Orchestration          L19  Caching & cost
 L17  Evaluation             L20  Observability
 L18  Governance & security  L21  Serving & freshness

AGENTIC-ONLY
```

```
L22  Planning & decomposition      L25  Critique & self-correction
L23  Tool registry & selection     L26  Termination & budget control
L24  Memory tiers                  L27  Multi-agent topology
```

PART 1 — LAYER BY LAYER

L0 — Data sources and connectors

Source type	Options	Good for	Watch out for
File stores	S3, Azure Blob, GCS, SharePoint, Google Drive, NFS	Bulk document corpora	Permissions rarely travel with the file; you must fetch them separately
Wikis / knowledge bases	Confluence, Notion, SharePoint, MediaWiki	Policies, runbooks, internal docs	Heavy boilerplate; nested page hierarchies matter as metadata
Ticketing / CRM	Jira, ServiceNow, Zendesk, Salesforce, HubSpot	Support history, resolution patterns	High duplication; resolution quality varies wildly
Comms	Slack, Teams, email (Exchange/Gmail)	Tribal knowledge	Extremely noisy; strong privacy constraints; threading is the useful unit
Databases / warehouses	Postgres, MySQL, Snowflake, BigQuery, Databricks	Structured facts, metrics	**Usually should NOT be embedded — route to SQL instead**
Code	GitHub, GitLab, Bitbucket	Developer assistance	Needs AST-aware handling, not text handling
Web	Sitemaps, crawlers, RSS	Public docs, competitor info	Legal/ToS constraints; content drift
APIs	Any REST/GraphQL endpoint	Real-time state (order status, inventory)	**Never index — call live at query time**

Connector tooling: Airbyte, Fivetran, Unstructured.io connectors, LlamaHub (LlamaIndex), LangChain document loaders, Apache NiFi, custom SDK pulls. **MCP servers** are increasingly the standard interface for tool/data access in agentic systems.

Decision rule: if the data changes faster than your reindex cycle, it's an API call, not an index entry.

L1 — Parsing and extraction

This layer causes more RAG failures than any other. Budget time here accordingly.

PDF

Approach	Tool	Good for	Weak at
Fast text extraction	PyMuPDF (fitz), pdfplumber, pypdf	Clean, single-column digital PDFs	Multi-column (reads across columns), tables, scans
Layout-aware open source	Unstructured, Docling, Marker, LayoutParser	Mixed real-world documents; detects titles, tables, figures	Slower; some need GPU for best results
Managed document AI	LlamaParse, Azure AI Document Intelligence, AWS Textract, Google Document AI, Reducto, Chunkr	High-accuracy tables and forms at scale	Cost per page; data leaves your perimeter
OCR	Tesseract, PaddleOCR, cloud OCR	Scanned documents	Error rate degrades silently — measure it
Vision-model extraction	Any multimodal LLM, GPT/Claude/Gemini vision	Messy scans, handwriting, complex layout	Expensive per page; can hallucinate table values
Skip parsing entirely	ColPali / ColQwen page-image embedding	Charts, diagrams, visual-heavy documents	Large storage; needs multi-vector index support

Everything else

Format	Tool	Note
DOCX	python-docx, Unstructured, Pandoc	Tracked changes, comments, footnotes are separate streams — decide explicitly
XLSX / CSV	pandas, openpyxl, Polars	**Decide first: is this prose or is this a dataset?** See below
PPTX	python-pptx, Unstructured	Speaker notes are often the real content
HTML	Trafilatura, readability-lxml, BeautifulSoup, Jina Reader	Strip nav, sidebars, cookie banners or they match every query
JSON / XML	Custom traversal, jq	Don't stringify raw — flatten to natural-language statements
Email	mailparser, Python `email`, Unstructured	Thread as the unit, not the message; strip quoted replies
Code	tree-sitter, LSP, AST parsers	Never split code by character count
Audio	Whisper, WhisperX, Deepgram, AssemblyAI, Speechmatics	**Require diarisation + timestamps** — they're how you cite
Video	ffmpeg keyframes + ASR + on-screen OCR	Three streams; OCR of slides is often the richest
Images	Vision LLM captioning, CLIP/SigLIP embeddings, OCR	Caption for structure, not aesthetics

The spreadsheet decision (most commonly botched)

Strategy	How	Use when
Row-as-sentence	Each row → natural language using column headers	Lookup questions: "what's the warranty on SKU X?"
Sheet-as-table	Convert to Markdown/HTML table, keep whole with a caption	Small reference tables, comparison matrices
Load to SQL	Put it in a real table, use text-to-SQL	**Any aggregate question — count, sum, average, group by**

> **Rule: vector search cannot count. If the questions are arithmetic, it's a SQL problem,**

> **not a retrieval problem.**

L2 — Cleaning and normalisation

Task	Options / tools	Why it matters
Boilerplate removal	Regex, Trafilatura, repeated-block detection	Headers/footers repeat into every chunk, match every query, and you pay for them forever
Exact deduplication	SHA-256 content hash	`policy_final_FINAL_v2.docx` fills your top-k with the same content
Near-duplicate detection	MinHash/LSH (datasketch), SimHash, embedding cosine > 0.97	Same content, minor edits — the more common case
Version resolution	`is_current` flag, effective-date metadata	Prevents quoting superseded policy
Language detection	fastText langdetect, lingua	Route to the right embedding model; affects token budgets
Unicode normalisation	NFKC normalisation	Smart quotes, ligatures, zero-width chars break exact matching
Table-of-contents / index page removal	Heuristic, page-position rules	TOC pages are keyword soup that match everything
PII detection & redaction	Microsoft Presidio, AWS Comprehend, Google DLP, spaCy NER	Compliance; and PII in chunks leaks through citations
Hidden-text stripping	Extract invisible/white-on-white text, zero-width chars	**Security** — this is where indirect prompt injection hides

---

L3 — Metadata and access control capture

Capture at ingestion or lose it forever. Retrofitting is painful and usually incomplete.

Metadata to always capture

`source_system` · `document_id` · `document_title` · `url` · `version` · `is_current`
`page_number` · `section_path (H1 > H2 > H3)` · `author` · `created_at` · `modified_at`
`document_type` · `department` · `language` · `confidentiality_label`
`ingestion_run_id` · `content_hash` · `chunk_id` · `chunk_index`

Access control patterns

Pattern	How it works	Good for	Cost
Chunk-level ACL list	Store allowed groups/users on each chunk; pre-filter at query time	Most enterprise cases	Index size; sync lag on permission changes
Live permission check	Retrieve candidates, then verify each against the source system	Highest accuracy, no sync lag	Latency; source-system load
Partitioned indexes	Separate index per security boundary	Strong isolation, simple reasoning	Duplication; cross-boundary queries impossible
Row-level security in the DB	Postgres RLS with pgvector	Elegant if you're already on Postgres	Only works in a SQL-backed store
Late-binding filter	Filter after retrieval	❌ **Avoid** — returns empty results, and is a leak waiting to happen	—

> **The citation leak:** a citation to a document title the user can't open is itself a disclosure. Filter citation metadata too.

L4 — Chunking

Strategy	How	Good for	Avoid when
Fixed-size	Split every N tokens	Baseline only	Anything real
Fixed + overlap	N tokens, 10–20% overlap	Uniform prose, fast start	Structured documents
Recursive	Split on `\n\n` → `\n` → `.`		

character	→ respecting max size	Solid general default	Tables, code
Document-structure aware	Split on headings/sections; carry the heading path into each chunk	**Best default for enterprise docs**	Unstructured narrative
Semantic chunking	Embed sentences, split where similarity drops	Flowing narrative with no headings	Costs an embedding pass; unpredictable sizes
Parent-child (small-to-big)	Embed small chunks, return the larger parent at retrieval	**Excellent, widely applicable** — precision in search, completeness in context	Slightly more storage complexity
Contextual retrieval	Prepend an LLM-generated context line to each chunk *before* embedding	**Best accuracy-per-effort available**	Needs one cheap LLM call per chunk at index time
Late chunking	Embed the whole doc with a long-context embedder, then pool into chunk vectors	Preserves cross-chunk context in the vectors	Requires a supporting embedding model
Hierarchical / RAPTOR	Recursively cluster + summarise into a tree; retrieve at multiple levels	Corpora needing both detail and whole-doc synthesis	Expensive to build and maintain
Proposition-based	Decompose into atomic standalone factual statements	Very high precision fact retrieval	Expensive; loses narrative flow
AST-based (code)	Split on function/class boundaries	**Any code corpus**	—
Row/record-based	One record per chunk	Tickets, CRM records, log entries	—
Speaker-turn / scene	Split on speaker change or scene cut	Audio, video, meeting transcripts	—

Starting parameters (then tune against an eval set)

```
Prose                    400–800 tokens, 10–15% overlap, structure-aware
Dense technical/legal    300–500 tokens  (precision matters more)
Support tickets          whole ticket if under ~1,000 tokens
Code                     one function/class per chunk
Tables                   never split; if forced, repeat the header row
Transcripts              one speaker turn or topic segment
```

Tools: LangChain text splitters, LlamaIndex node parsers, Chonkie, semantic-chunkers, `tree-sitter` for code, Unstructured's chunking-by-title.

L5 — Chunk enrichment

Cheap at index time, disproportionate payoff at query time.

Technique	What it adds	Payoff	Cost
Contextual prefix	One LLM-generated sentence: "This is from the FY25 revenue policy, section 4, on deferred revenue"	**Highest ROI enrichment there is**	One cheap LLM call per chunk (cache the document to cut this heavily)
Heading path injection	Prepend `Policies > Finance > Revenue Recognition > 4.2`	Nearly free; big gain on structured docs	Zero
Generated title	LLM writes a title per chunk	Improves both retrieval and UI display	One call per chunk
Hypothetical questions	Generate 3–5 questions this chunk answers; embed those too	Closes the question↔document asymmetry gap	Extra vectors to store
Chunk summary	Short abstract embedded alongside the full text	Good for long chunks	One call per chunk
Keyword / entity extraction	Pull named entities, product codes, dates into metadata	Enables filtering and boosts hybrid search	Cheap (NER) or moderate (LLM)
Auto-classification	Tag department, doc type, sensitivity	Powers routing and filtering	Cheap classifier

Table caption generation	Describe what a table contains	Tables are otherwise near-unsearchable	One vision/LLM call

---

L6 — Embedding

Model families

Family	Examples	Good for	Note
Commercial dense	OpenAI `text-embedding-3-small/large` , Cohere Embed v4, Voyage `voyage-3` , Google Gemini embedding	Strong general quality, zero ops	Per-token cost; data leaves perimeter
Open-weight dense	BGE-M3, E5-Mistral, GTE, Nomic Embed, Jina v3, Qwen3-Embedding	Self-hosting, data residency, fine-tuning	You own the GPU and the ops
Multilingual	BGE-M3, multilingual-E5, Cohere multilingual, Jina	Non-English or mixed corpora	Check your specific language, not the average
Domain-specific	BioBERT/PubMedBERT-derived, legal-domain models, code embedders	Heavy jargon domains	Narrower general performance
Sparse / learned sparse	BM25 (statistical), SPLADE, BM42	Exact terms, codes, rare entities	Pair with dense, don't replace
Multi-vector / late interaction	ColBERT (via RAGatouille), ColPali, ColQwen	Highest accuracy; visual documents	~100–1,000 vectors per item — storage heavy
Multimodal	Cohere Embed v4, `voyage-multimodal-3` , CLIP/SigLIP	Text + image in one vector space	Coarser than late interaction for documents
Code	`voyage-code` , Qwen coder embeddings, `jina-embeddings-v2-base-code`	Code search	Pair with AST

			chunking

Dimensions

```
384         Fast, small, weakest       — high-volume / edge
768         Good balance               — most workloads
1024–1536   Standard production tier
3072+       Maximum quality            — rerank stage, or small corpora

Matryoshka models (OpenAI v3, Nomic, Jina): truncate to 512 for first-pass
ANN search, keep full dimensions for rescoring. ~6× storage cut, few points
of recall lost.
```

The three configuration bugs that silently destroy recall

1. **Metric mismatch** — index configured for cosine, model trained for dot product (or vice versa).
2. **Missing query prefix** — asymmetric models expect `"query: "` / `"passage: "` or an instruction prefix. Omitting it costs real recall and produces no error.
3. **Model drift** — you re-embed queries with a newer model version than the index was built with. **Version-pin your embedding model and store the version in metadata.**

L7 — Compression and quantisation

Method	Size reduction	Recall impact	Use when
float32 (none)	1×	baseline	Under ~1M vectors
float16 / bfloat16	2×	negligible	Always safe
int8 scalar	4×	usually <1%	**Default for production at scale**
Product Quantisation (PQ)	8–32×	noticeable	Very large corpora with rescoring
Binary	32×	large alone	Only with a rescoring stage
Matryoshka truncation	2–6×	small	Models that support it — combine with int8

MUVERA (multi-vector → fixed dim)	large	small	Making ColBERT/ColPali affordable

The standard large-scale pattern: binary or int8 first-pass over a wide candidate set → rescore the top few hundred with full-precision vectors → rerank the top 50 with a cross-encoder.

L8 — Indexes and storage

Index types

Index	Mechanism	Knobs	Character
Flat (brute force)	Compare against everything	none	Perfect recall. Fine under ~50K vectors. **Use as your accuracy baseline.**
HNSW	Navigable small-world graph	`M`, `efConstruction`, `efSearch`	Industry default. Fast, high recall, RAM-hungry.
IVF / IVF-PQ	Cluster, search nearest clusters only	`nlist`, `nprobe`	Memory-efficient; needs a training step; recall sensitive to `nprobe`
DiskANN / SPANN	Graph index designed for SSD	varies	Billion-scale on commodity hardware
ScaNN	Anisotropic quantisation + partitioning	varies	Google-scale, strong throughput

Vector stores

Store	Best for	Trade-off
pgvector / pgvectorscale (Postgres)	**Default enterprise choice under ~10M vectors.** One system, transactional, ACLs beside your data, RLS available	Scaling limits; hybrid search needs extra work
Qdrant	Strong filtering, native hybrid, sparse + multi-vector support, good Rust performance	Another system to operate

Weaviate	Native hybrid search, built-in modules, GraphQL	Opinionated
Milvus / Zilliz	Billion-scale, many index types	Operationally heavy
Elasticsearch / OpenSearch	**You already have BM25** — hybrid in one system	Vector performance trails dedicated engines
Vespa	Most sophisticated ranking; late interaction; huge scale	Steepest learning curve
Pinecone	Fully managed, zero ops	Cost; vendor lock-in
Chroma / LanceDB / FAISS	Prototyping, embedded, local	Not production-grade at scale (FAISS is a library, not a DB)
Redis / Valkey	Already in your stack; low latency	Memory cost
Turbopuffer	Object-storage backed, very cheap at scale	Higher cold latency
MongoDB Atlas / Azure AI Search	Already on that platform	Platform-bound

Filtering strategy — the subtle correctness issue

Approach	Behaviour	Verdict
Post-filter	Search top-k, then drop non-matching	❌ Can return zero results
Pre-filter	Filter the set, then search it	Slow on large filtered subsets
Filtered ANN	Apply the predicate *during* graph traversal	✅ **What you want. Ask your vendor if they do this.**

Companion indexes you need alongside vectors

Index	Purpose	Tools
Inverted / BM25	Exact terms, codes, names	Elasticsearch, OpenSearch, Lucene, Tantivy, `rank_bm25` , Postgres FTS
Knowledge graph	Multi-hop, relationships	Neo4j, Memgraph, Kùzu, NebulaGraph, Microsoft GraphRAG, LlamaIndex property graph

SQL / warehouse	Aggregates, metrics, counts	Postgres, Snowflake, BigQuery, DuckDB
Document store	Full original text for parent expansion	Postgres, S3, MongoDB, any KV store
Metadata store	Filters, ACLs, lineage	Same DB as above — keep it transactional

---

L9 — Query understanding

The query the user typed is rarely the query you should search with.

Technique	What it does	Good for	Cost
Retrieval gating	Decide whether to search at all	"hi", "thanks", "rephrase that" don't need a search. Cuts latency and cost in chat	One cheap classifier call
Query rewriting	Resolve pronouns/ellipsis from history: "and last year?" → "ACME FY24 revenue?"	**Mandatory for multi-turn. Skipping this is the #1 chat-RAG bug**	One cheap call
Multi-query expansion	Generate 3–5 paraphrases, retrieve each, fuse	Vague or under-specified questions	N× retrieval cost
Decomposition	Break a compound question into sub-questions	"Compare X and Y on Z"	N× retrieval
HyDE	LLM writes a *hypothetical answer*, embed that, search with it	Closes the question↔document shape gap	One call, adds latency
Step-back prompting	Ask a broader question first for conceptual grounding	Reasoning-heavy questions	One extra retrieval
Filter extraction	Pull structured constraints from natural language: "legal docs updated after January" → `dept=legal AND modified>2026-01-01`	**Large precision gain, cheap**	One structured-output call

Query routing / classification	Decide which index or tool handles this	**Highest-leverage component in hybrid systems**	One cheap classifier
Spelling / acronym normalisation	Expand internal acronyms, fix typos	Enterprise jargon	Dictionary lookup
Query intent classification	lookup / aggregate / comparison / action	Drives routing and model tier selection	Cheap classifier

---

L10 — Retrieval

Method	Finds	Blind to	Tools
Dense / vector	Meaning, paraphrase, synonyms	Exact codes, numbers, negation, rare entities	Any vector DB
Sparse / BM25	Exact terms, IDs, names, error codes	Synonyms, paraphrase	Elasticsearch, OpenSearch, Lucene, Postgres FTS, `rank_bm25`
Learned sparse (SPLADE/BM42)	Exact terms + learned expansion	Heavier index	Qdrant (BM42), Pinecone sparse, Elasticsearch ELSER
Hybrid (dense + sparse)	**Both. The correct default.**	—	Qdrant, Weaviate, Vespa, Elasticsearch, or do it yourself
Multi-vector / late interaction	Token-level precision; visual layout	Storage cost	ColBERT/RAGatouille, Vespa, Qdrant multi-vector, ColPali
Graph traversal	Multi-hop relationships	Needs graph construction	Neo4j, Microsoft GraphRAG, LlamaIndex property graph
Text-to-SQL	**Counts, sums, averages, group-by**	Unstructured prose	Vanna, LlamaIndex NLSQL, LangChain SQL agent, DB-native
			Tool/function calling, MCP

Live API call	Current state	Nothing is indexed	servers
Metadata-only filter	Known-attribute lookup	Semantics	Any DB
Parent-child expansion	Precise match, full context returned	—	LlamaIndex auto-merging, LangChain ParentDocumentRetriever
Recursive / iterative	Retrieve → notice gap → retrieve again	Latency	LangGraph, custom loop

Retrieval depth

```
First pass (recall-oriented)        k = 50–100
After fusion                        k = 50
After reranking (precision)         k = 3–10  → this is what reaches the model
```

> **Retrieve broadly for recall, then narrow aggressively for precision. Recall is unrecoverable downstream; precision is fixable by reranking.**

L11 — Fusion

Method	How	Verdict
Reciprocal Rank Fusion (RRF)	`score = Σ 1/(60 + rank)` across lists	✅ **Start here.** No score normalisation needed, robust, one line of code
Weighted score blending	`α × dense + (1−α) × sparse`	Requires normalising incompatible score scales. Tunable but fragile
Relative score fusion	Min-max normalise each list, then blend	Middle ground; Weaviate does this natively
Cascade / fallback	Try sparse; if weak results, try dense	Cheap; loses the agreement signal
Learned fusion	Train a model on click/relevance data	Best if you have the data; most don't

Why RRF works: it operates on *ranks*, not scores. A cosine similarity of 0.87 and a BM25 score

of 14.3 are not on the same scale and cannot be sensibly added. Ranks are always comparable.

L12 — Reranking

The single highest-ROI addition to any working RAG system. If someone says "our RAG is 70% accurate," this is the first thing to try.

Method	How	Accuracy	Cost/latency
Cross-encoder reranker	Scores query + document *together*	**+10–25 nDCG typical**	~50–200ms for 50 docs, fractions of a cent
LLM-as-reranker	Ask an LLM to rank the candidates	Good, flexible, explainable	10–100× more expensive than a dedicated reranker
ColBERT late interaction	MaxSim over token vectors	Very high	Storage-heavy
MMR (Maximal Marginal Relevance)	Trades relevance for diversity, λ ≈ 0.5–0.7	Prevents five paraphrases of one paragraph	Near-free
Business-rule boosting	Boost by recency, source authority, document status	Essential in enterprise — an approved policy should outrank an old Slack thread	Free
Diversity by source	Cap chunks per document	Stops one document dominating top-k	Free

Tools: Cohere Rerank, Voyage rerank, Jina Reranker, BGE-reranker (open), mixedbread rerank, `cross-encoder/ms-marco-MiniLM` (open, small, fast), Vespa/Elasticsearch native rerank stages.

Rule: rerank the top ~50, not the top 500. Cost scales linearly with candidate count and returns flatten fast.

L13 — Context assembly and compression

Technique	Purpose	Note
Deduplication	Drop near-identical chunks	Cheap; do it always
Relevance thresholding	Drop everything below a score floor	**If nothing clears the floor, refuse.** Eliminates a whole class of confident hallucination
Position ordering	Best evidence first *and* last	Counters "lost in the middle"
Parent expansion	Swap matched child chunks for their parent sections	Precision in search, completeness in context
Contextual compression	Cheap model extracts only query-relevant sentences from each chunk	Large token savings on long chunks; adds a call
Token budgeting	Hard cap on assembled context	Prevents runaway cost
ACL re-check	Enforce permissions again, here	**Defence in depth — retrieval bugs happen**
Citation binding	Attach chunk ID → source + page to each passage	Non-negotiable for verification and compliance
Structured framing	Wrap in `<document id=... source=...>` tags	Improves grounding *and* is an injection defence

Tools: LangChain ContextualCompressionRetriever, LLMLingua (prompt compression), LlamaIndex postprocessors, custom.

L14 — Generation

Element	Options	Note
Grounding instruction	"Answer only from `<context>` " + explicit refusal token	The single highest-value line in a RAG prompt
Citation format	Inline `[doc-1]` , footnotes, structured JSON `sources[]`	Structured is best for programmatic verification
Refusal	`INSUFFICIENT_CONTEXT` sentinel	Turns silent failure into a countable

path		metric
Output format	Prose / JSON schema / enum / markdown	Native structured-output mode beats "please return JSON"
Model tier	Cheap for extraction, mid for RAG answering, flagship for analysis	Retrieval did the hard part — mid tier handles most RAG answering
Reasoning effort	Off / low / high	Billed as output tokens. Buy deliberately.
Streaming	Token-by-token	Time-to-first-token is what users perceive
Prefill	Start the response with `{` or `<answer>`	Forces format, skips preamble

---

L15 — Post-generation verification

Skipped by most teams. This is where trust is actually built.

Check	How	Catches
Groundedness / faithfulness check	Second cheap call: "is every claim supported by the context?"	Residual hallucination
Citation validation	Verify each cited chunk actually supports the claim	Fabricated or mismatched citations
Schema validation	JSON schema / Pydantic	Malformed structured output
Numeric / fact cross-check	Re-extract numbers from source and compare	Transcription errors on figures
PII / secrets scan on output	Presidio, regex, DLP	Leakage
Policy guardrails	NeMo Guardrails, Guardrails AI, Llama Guard	Toxicity, off-topic, policy violations
Exfiltration pattern scan	Look for encoded data in URLs, unexpected tool calls	Successful prompt injection
Confidence signal to	Show source count, scores, or a "low	Lets users calibrate

UI	confidence" flag	trust

L16 — Orchestration

What this layer owns

```
Request lifecycle · state & memory · tool registry & permissions
retries, timeouts, fallbacks · routing · tracing · cost accounting
```

Approach	Tools	Good for	Trade-off
Full framework	LangChain / LangGraph, LlamaIndex, Haystack, Semantic Kernel	Fast prototyping, graph-structured control flow	Abstraction overhead; debugging through layers
Agent frameworks	LangGraph, CrewAI, AutoGen, OpenAI Agents SDK, Pydantic AI, Google ADK	Multi-agent, stateful loops	Non-determinism; rapid API churn
Declarative / optimising	DSPy	Programmatic prompt optimisation against metrics	Different mental model; smaller community
Durable workflow engines	Temporal, Restate, AWS Step Functions	Long-running agents that must survive restarts	Infrastructure weight
Custom code + SDKs	Direct provider SDKs + your own orchestration	**What most teams converge on for the hot path**	You build retries, tracing, state yourself
Tool interoperability	MCP (Model Context Protocol) servers	Standardised tool/data access across agents	Newer; vet third-party servers

Engineering practices that matter more than framework choice:

- Every component behind an interface — embedder, store, reranker, LLM all swappable
- Config-driven parameters (chunk size, k, thresholds, model IDs) so tuning doesn't need a deploy

- **Distributed tracing on every request**, capturing: rewritten query, retrieved chunk IDs *with scores*, reranked set, final assembled prompt, full token usage
- Circuit breakers per provider; a cheaper model as fallback; graceful degradation (return the documents with an apology rather than nothing)
- Idempotency keys and a dead-letter queue for ingestion failures

> **Without the trace, RAG is undebuggable. With it, most bugs are obvious in thirty seconds.**

L17 — Evaluation

Retrieval metrics (measure these separately from generation)

Metric	Formula	Target
Recall@k	relevant retrieved ÷ total relevant	**>0.90 at k=20 before reranking. The most important RAG metric.**
Precision@k	relevant retrieved ÷ k	>0.7 at k=5 after reranking
MRR	mean of 1/(rank of first relevant)	When one correct answer exists
nDCG@k	rank-discounted graded relevance	Standard for comparing rerankers
Hit rate@k	fraction of queries with ≥1 relevant in top k	Simplest go/no-go

Generation metrics

Metric	Question
Faithfulness / groundedness	Is every claim supported by the context? *(primary hallucination metric)*
Answer relevance	Does it address what was asked?
Answer correctness	Does it match ground truth?
Context precision	Are the useful chunks ranked highest?

Context recall	Does the context contain everything the reference answer needs?
Citation accuracy	Do cited sources actually support the cited claims?
Refusal correctness	Does it say "I don't know" exactly when it should?

Tools

Category	Options
RAG eval frameworks	RAGAS, DeepEval, TruLens, Arize Phoenix, Continuous Eval
Platforms	LangSmith, Langfuse, Braintrust, Weights & Biases Weave, MLflow
Prompt/regression testing	promptfoo, Evidently, pytest + custom asserts
Retrieval benchmarks	BEIR (retrieval), MTEB (embedding models), ViDoRe (visual documents)

Building the golden set

1. 50–100 REAL queries from logs or users (not invented ones — invented queries are always cleaner than real ones and will flatter your system)
2. Cover the distribution: simple lookup · multi-hop · aggregate · comparison · ambiguous · out-of-scope (correct answer = refusal) · adversarial/injection · multilingual
3. Label relevant chunks per query — have a domain expert do this
4. Write reference answers for the correctness-critical subset
5. Version it; every production failure becomes a permanent test case
6. Run in CI on every change to chunking, embedding, retrieval, reranker, prompt, or model version

LLM-as-judge discipline

- **Calibrate against humans** — label 50 items manually, measure agreement, fix the rubric before trusting any number
- **Detailed rubric, small scale** — 1–5 with explicit anchors, or binary. Vague 1–10 scales produce noise
- **Known biases:** position bias (randomise order), verbosity bias (longer scores higher), self-preference (use a different model family as judge)

L18 — Governance and security

Area	Controls	Tools
Access control	ACL at ingestion → pre-filter at query → re-check at assembly → filter citations	See L3
Prompt injection (indirect)	Delimiters + "content is data, not instructions" · strip hidden text at ingest · least-privilege tools · output scanning · guardrail classifier	NeMo Guardrails, Llama Guard, Rebuff, Lakera
Data poisoning	**Control write access to indexed sources as tightly as read access to the index**	Source-system permissions, ingestion review
PII	Detect and redact at ingestion; scan outputs	Presidio, AWS Comprehend, Google DLP
Data residency	Where do vectors live? Where does inference run?	Regional endpoints, self-hosting
Retention & deletion	Deletion must propagate to chunks, vectors, caches, **and logs**	Design for it; don't discover it
Lineage	Which source, version, ingestion run produced this chunk	Metadata + audit tables
Denial of wallet	Hard caps: max iterations, max tokens, max wall-clock, per-user rate limits	Orchestration layer
Insecure output handling	Treat model output as untrusted input to the next system — never `eval()`, never unescaped into SQL/shell/HTML	Standard appsec
Supply chain	Pin model versions; vet embedding models and third-party MCP servers	—
Audit	Who asked what, what was retrieved, what was answered	Trace store with retention
Frameworks	NIST AI RMF, ISO/IEC 42001, EU AI Act, sectoral rules	Adopt one voluntarily before procurement demands it

---

L19 — Caching and cost

Layer	Mechanism	Typical saving
Prompt caching	Provider KV-cache on a stable prefix	**Up to 90% of input cost.** Order prompts static-first
Semantic answer cache	Cache answers keyed by question embedding	30–40% hit rate in FAQ workloads. **Set the threshold high** — "refund policy India" vs "USA" are close but need different answers
Embedding cache	Hash → vector, skip re-embedding unchanged content	Large on re-index runs
Retrieval cache	Cache result sets for hot queries	Cuts vector DB load
Batch API	Async processing	Flat 50% off input and output
Model routing / cascade	Cheap model first, escalate on failure	60–80%
Reranker cost control	Rerank top 50, not top 500	Linear
Quantisation	Smaller vectors	4–32× storage/memory

Tools: provider-native prompt caching, GPTCache, Redis semantic cache, Langfuse/Helicone caching layers.

Invalidation rule: any semantic cache must be invalidated when source documents change. Otherwise you're serving answers from a policy that no longer exists.

L20 — Observability

Log per request, without exception:

```
model_id · input_tokens · cached_tokens · output_tokens · reasoning_tokens
latency_ms (per stage) · retrieval_k · top_scores · reranked_ids
escalated · request_type · user_id · refusal_flag · trace_id
```


Signal	What it tells you
`INSUFFICIENT_CONTEXT` rate	Direct retrieval-quality meter
Zero-result / low-score rate	Index gaps
Citation click-through	Implicit relevance signal
Escalation rate (cascade)	Live quality drift indicator
Query distribution drift	New topics appearing = coverage gaps = your roadmap
Cost per request by type	**Alert on this, not total spend**
p50/p95 latency per stage	Where the time actually goes

Tools: LangSmith, Langfuse, Arize Phoenix, Helicone, OpenLLMetry / OpenTelemetry GenAI conventions, Datadog LLM Observability.

L21 — Serving and freshness

Concern	Options
Incremental indexing	Content hash → upsert on change, tombstone on delete. **Design for this from day one** — full reindex is fine at 10K docs and impossible at 10M
Change data capture	Source webhooks, polling, DB CDC (Debezium), scheduled diffs
Freshness SLA per source	Tickets: minutes. Policies: daily. Archives: weekly
Index versioning	Blue/green indexes so you can roll back an embedding-model change
Backfill strategy	Re-embedding 10M chunks is a project, not a task — plan it
Latency budget	Embed 80ms · ANN 25ms · rerank 180ms · TTFT ~900ms → ~1.3s total
Parallelism	Run dense + sparse retrieval concurrently; parallelise independent tool calls
Fallbacks	Cheaper model, cached answer, or return raw documents with an honest

	message

PART 2 — THE MATURITY MATRIX

What you actually pick at each layer, by system maturity.

Layer	**Basic RAG**	**Advanced RAG**	**Hybrid RAG**	**Agentic RAG**
Sources	One corpus, one format	Several formats, one pipeline	Many sources, per-source pipelines	Many sources + live APIs + tools
Parsing	Basic text extraction	Layout-aware + OCR	+ tables to SQL, visual pages to image index	+ on-demand parsing at query time
Cleaning	Minimal	Boilerplate + dedup + versioning	+ per-source normalisation	Same
Metadata	Source + page	Full schema + heading path	+ source trust weighting	+ tool/action metadata
ACLs	Often skipped ⚠️	Chunk-level ACL, pre-filtered	+ per-source ACL propagation	+ per-tool permission scoping
Chunking	Fixed + overlap	Structure-aware + parent-child	Per-source strategies	Same, + dynamic re-chunking
Enrichment	None	**Contextual prefix** + heading path	+ generated questions, table captions	+ summaries for agent planning
	One general	+ correct prefixes,	+	

Embedding	model	version pinned	domain/multilingual/multimodal models	Same
Compression	None	int8	int8 + Matryoshka	Same
Index	HNSW, one index	HNSW + filtered ANN	Vector + BM25 + graph + SQL	+ live tool endpoints
Query understanding	None — raw query	**Rewriting** + filter extraction	+ **routing across backends**	Agent decomposes and plans
Retrieval	Dense top-k	Dense, tuned k	**Hybrid + RRF**, multi-backend	Iterative, agent-selected per step
Fusion	n/a	n/a	**RRF**	RRF + agent judgement
Reranking	None	**Cross-encoder**	+ business rules, MMR, source trust	+ agent relevance critique
Assembly	Concatenate	Dedup, threshold, ordering	+ parent expansion, compression	Dynamic budget per step
Generation	Basic prompt	**Grounding + citations + refusal**	Same	+ synthesis across sources
Verification	None	Schema validation	+ groundedness check	**Self-critique loop**
Orchestration	Linear script	Linear + config-driven	Router + parallel retrievers	**Graph/loop with state**
Evaluation	Eyeball it ⚠️	**Golden set in CI, recall@k**	+ per-route metrics	+ trajectory & tool-use eval
Cost control	None	Caching + output limits	+ routing by backend	+ **hard iteration/token caps**

Typical latency	0.5–1s	0.8–1.5s	1–2s	5–30s
Relative cost/query	1×	1.1×	1.3×	5–10×

The rule that governs this table

> **Earn your way up. An agent does not repair bad retrieval — it amplifies it.** With a poor retriever, an agent fails five times instead of once, at five times the cost. The most expensive mistake in this field is jumping to Agentic when the actual defect was in chunking.

PART 3 — AGENTIC-ONLY LAYERS

These don't exist in Basic/Advanced/Hybrid. They're what make agentic systems different — and harder.

L22 — Planning and decomposition

Pattern	How	Good for
ReAct	Interleave reasoning and actions	The foundational loop. Start here.
Plan-and-execute	Generate a full plan first, then execute steps	Predictable cost; plan survives a failed step
Least-to-most	Solve simplest sub-problem first, build up	Dependent reasoning chains
Tree-of-thought	Explore multiple branches, prune	High-stakes, expensive
Adaptive routing	Classify complexity → direct answer / single retrieval / full loop	**Biggest cost saver in agentic RAG**

L23 — Tool registry and selection

Concern	Options

Tool definition	JSON schema function calling; MCP servers
Tool count problem	20 tools ≈ 5,000 input tokens **per call**. Prune per route, or use lazy tool search
Selection	Model-chosen / router-chosen / rule-gated
Permissions	**Blast radius = permissions of the tools the agent can call.** Read-only agent is a nuisance; write-capable agent is an incident
Destructive actions	Gate behind human confirmation, always

L24 — Memory tiers

Tier	Contents	Implementation
Short-term	Current conversation	Sliding window / compaction
Working	Scratchpad for the current task	**Offload to files/tools rather than carrying in-context**
Long-term	Facts, preferences, past resolutions	RAG over your own interaction history
Procedural	Learned workflows, successful trajectories	Trajectory store, few-shot injection
Episodic	"What happened in session 47"	Summarised session store

Tools: Mem0, Zep, Letta (MemGPT), LangGraph checkpointers, or plain Postgres + a vector index.

L25 — Critique and self-correction

Pattern	Mechanism
Self-RAG	Reflection tokens decide whether to retrieve and whether passages are relevant/supported
Corrective RAG (CRAG)	Grade retrieved docs; if poor, fall back to another source (often web)

Reflexion	Verbal self-feedback after a failed attempt, retried
Verifier model	Separate cheap model checks the output before returning
Grounding gate	Block the answer if groundedness score is below threshold

L26 — Termination and budget control

Non-negotiable. Every one of these, every time.

```
max_iterations          (typically 5–10)
max_total_tokens        (hard ceiling per request)
max_wall_clock_seconds
required_termination_path   ("if you cannot answer, say so and stop")
loop_detection              (same tool + same args twice → break)
per_user_rate_limit
cost_ceiling_per_request
```

L27 — Multi-agent topology

Topology	Shape	Good for	Risk
Single agent + tools	One loop, many tools	**Start here. Most tasks never need more.**	—
Supervisor / worker	Planner delegates to specialists	Clear separation, parallelisable	Supervisor becomes a bottleneck
Sequential pipeline	Fixed agent chain	Predictable, testable	Rigid
Parallel fan-out + synthesise	Multiple retrievers at once, one synthesiser	Multi-source research	Cost multiplies
Debate / critic	One generates, one critiques	High-stakes accuracy	2×+ cost
Hierarchical	Agents spawning agents	Very complex tasks	Cost and debugging explode

> Use the smallest topology that works. Multi-agent systems have the coordination and cost-

> attribution problems microservices had in 2016 — and the same tendency to be adopted before they're needed.

PART 4 — WHAT PEOPLE MISS

Layers and concerns that are not optional but get skipped anyway. You asked what was missing — this is the list.

#	Missed thing	Why it's skipped	What it costs you
1	**ACL capture at ingestion**	Feels like a "phase 2" concern	The single most common reason enterprise pilots never reach production. Retrofitting is painful and usually incomplete.
2	**Deletion propagation**	Nobody tests the delete path	Your assistant quotes a retracted policy. That's a compliance incident, not a bug.
3	**The evaluation set**	Feels like overhead before launch	You cannot safely change *anything* — model, chunk size, prompt — because you can't tell if it helped
4	**Table extraction quality measurement**	Extraction "looks fine" on inspection	Highest-error step in most pipelines, and errors are invisible downstream
5	**Query rewriting for multi-turn**	Works fine in single-turn testing	Every follow-up question ("and last year?") retrieves garbage
6	**Indirect prompt injection**	You control the prompt, so it feels safe	You deliberately feed untrusted documents to an instruction-follower. Hidden text in a PDF is the attack surface.
7	**Write-access control on indexed sources**	Everyone secures the index, nobody secures the sources	Anyone who can edit an indexed wiki page can plant content designed to be retrieved

8	**Incremental update path**	Full reindex works at prototype scale	Fine at 10K docs, impossible at 10M. Retrofitting CDC is a rewrite.
9	**Embedding model version pinning**	The model "just works"	A silent provider update re-embeds queries in a different space than the index. Recall collapses with no error.
10	**The retrieval trace**	Logging feels like polish	Without retrieved chunk IDs + scores per request, RAG is undebuggable
11	**Cost per request attribution**	Total spend looks fine	Unit cost doubles behind flat traffic and nobody notices for weeks
12	**Refusal path +** `INSUFFICIENT_CONTEXT`	Seems like admitting defeat	Without it, retrieval failures become confident hallucinations instead of a countable metric
13	**Source trust weighting**	All chunks treated equally	An approved policy ranks below a two-year-old Slack thread
14	**Aggregate-question routing**	"It's all just questions"	Vector search cannot count. You get confidently wrong totals.
15	**Whole-document summarisation path**	Assumed RAG handles it	"Summarise this 200-page report" returns five paragraphs, not a summary
16	**Retrieval gating**	Always retrieving seems safer	You pay for a vector search on "thanks" and "can you rephrase that"
17	**Chunk-level duplicate suppression**	Dedup done at document level only	Top-5 fills with five copies of the same paragraph from different documents
18	**Language-specific token budgets**	Costed in English	Non-English content costs 2–5× for the same information
19	**Blue/green index versioning**	Index changes done in place	No rollback when an embedding-model change degrades quality
20	**Human handoff path**	Assumed the system always answers	No escalation route when the system genuinely can't help
21	**Feedback capture with reason codes**	Thumbs up/down with no	You know something's wrong but not what

		"why"	
22	**Cold-start / empty-corpus behaviour**	Tested with full data	Day one in a new deployment returns nothing, confidently
23	**Conflicting-sources handling**	Assumed one truth exists	Two documents disagree; the model silently picks one. It should surface the conflict.
24	**Chunk-size / model-swap regression testing**	Parameters tuned once, never revisited	Every parameter you tuned for last year's model may be wrong for this year's
25	**Tool definition token cost**	Invisible in the prompt you wrote	20 tools ≈ 5,000 tokens on every single call

---

PART 5 — QUICK PICKS

"My problem is X → do Y"

Symptom	Most likely layer	First thing to try
Right answer exists but isn't retrieved	**Chunking or parsing (L1/L4)**	Print the retrieved chunks. Check recall@20.
Right chunks retrieved but ranked low	**Reranking (L12)**	Add a cross-encoder reranker
Fails on error codes, part numbers, names	**Retrieval method (L10)**	Add BM25, fuse with RRF
Fails on follow-up questions	**Query understanding (L9)**	Add query rewriting from history
Confident wrong answers	**Generation (L14)**	Grounding instruction + refusal token + relevance threshold
Answers from outdated policy	**Freshness / versioning (L2, L21)**	Version flags + deletion propagation
Can't answer "how many /	**Routing (L9)**	Route aggregates to SQL

total"		
Multi-hop questions fail	**Architecture**	Graph retrieval or agentic loop
Top-5 is five copies of one paragraph	**Assembly (L13)**	Dedup + MMR
Great in demo, bad on real docs	**Parsing (L1)**	Layout-aware parser; inspect a multi-column page by hand
Cost too high	**L19**	Prompt caching → routing → batch → output limits
Too slow	**L21**	Parallelise retrieval, trim input, stream, lower reasoning effort
Quality degraded with no code change	**L6 / L21**	Check embedding-model version pinning and source drift
Leaks data across users	**L3 / L18**	ACL pre-filter in the ANN search; re-check at assembly

Minimum viable production stack

```
Parse           Unstructured or Docling (layout-aware)
Chunk           Structure-aware, 500 tokens, parent-child
Enrich          Contextual prefix (one cheap LLM call per chunk)
Embed           One good commercial or open model, version pinned
Store           Postgres + pgvector (if <10M vectors) or Qdrant
Also index      BM25 (Elasticsearch, OpenSearch, or Postgres FTS)
Query           Rewrite for multi-turn + extract filters
Retrieve        Hybrid dense + sparse, k=50, RRF
Rerank          Cross-encoder → top 5
Assemble        Dedup, threshold, best-first-and-last, ACL re-check
Generate        Mid-tier model, grounding instruction, citations, refusal token
Verify          Schema validation + groundedness check
Evaluate        50-query golden set in CI, recall@20 > 0.90
Observe         Full trace: query, chunk IDs, scores, prompt, tokens
Secure          ACLs at ingestion, pre-filtered; delimiters; least-privilege tools
```

The five parameters that matter most

```
1.  Chunk size              400–800 prose, 300–500 dense technical
2.  Retrieval k             50–100 first pass → 3–10 after reranking
3.  Overlap                 10–15%
```

4.	RRF constant	60 (leave it alone)
5.	Relevance threshold	tune on your eval set; below it, refuse

The four highest-ROI additions, in order

1.	Cross-encoder reranker	+10–25 nDCG, one afternoon
2.	Hybrid search (BM25 + RRF)	fixes the entire exact-match blind spot
3.	Contextual chunk prefix	cheapest large accuracy gain at index time
4.	Query rewriting	makes multi-turn work at all

Reference document. Tool landscape verified September 2026 — vendors and model names change; the layer structure does not.